CMSC 27200 - Spring 2025 — Lorenzo Orecchia
due 5/16/25 via Gradescope

H8.

- The assignment is due at Gradescope on 5/16/25.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using LaTeX. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work.* Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.
- Each student can request a total of up to 5 days of extensions on all homework - either for original submission or resubmission. Up to two days can be requested for a homework. If you would like to request an extension, please fill out the Homework extension form. Extensions do not carry any late penalty. If there is a life emergency that might prevent you from submitting on time not covered by this policy, email the Head TA.
- Taking into account the different focus students have coming into this course, you can choose between a more advanced theory problem or a programming assignment. **Complete either theory problem 4 or programming problem 5.**
- **Origin of Duality**: The concept of duality in linear programming was conceived in conversation between John von Neumann and George Dantzig. Von Neumann conjectured the theory, connecting it to his work in econimics, while Dantzig formalized the proof.

Problem 1 (Knapsack and Linear Programs) *Recall the 0-1 Knapsack problem discussed in class: we are given a set of items with values and weights: $\{(v_i, w_i)\}_{i=1}^n$ and a weight budget W, and we are interested in finding the choice of items (i.e. the subset S of $[n]$) maximizing the sum of the values of the items:*

$$\sum_{i \in S} v_i,$$

subject to the weight constraints:

$$\sum_{i \in S} w_i < W.$$

Consider the following linear program:

$$\begin{aligned} \max_{x \in \mathbb{R}^n} \quad & \sum_{i \in [n]} x_i v_i \\ & \sum_{i \in [n]} x_i w_i \leq W \\ & 0 \leq x_i \leq 1 \qquad \forall i \in [n] \end{aligned} \tag{1}$$

Here, each variable x_i is meant to indicate whether or not we are including a specific item in our solution.

(a) *Explain why the above linear program (1) doesn't solve the original 0-1 Knapsack problem, and provide an example instance of the 0-1 Knapsack problem that this LP gives an invalid solution for.*

(b) *Describe the problem, in words, that is solved by an instance of (1).*

(c) *Describe (in words) and give the pseudocode of an algorithm that finds the optimal assignment of x in (1).*

(d) *Compute the dual of (1).*

Collaborators:

Solution: Your solution here!

Problem 2 (A Flowy Metric) *Consider an undirected graph $G = (V, E)$ with capacities $c_e \geq 0$ on all edges $e \in E$. G has property that any cut in G has capacity at least 1. For example, a graph with a capacity of 1 on all edges is connected if and only if all cuts have capacity at least 1. However, in our case, c_e can be any non-negative number.*

(a) *Show that for any two vertices $s, t \in V$, the max flow from s to t is at least 1.*

(b) *Define the **length** of a flow f to be $\texttt{length}(f) = \sum_{e \in E} |f_e|$. Define the **flow distance** $d_{flow}(s, t)$ to be the minimum length of any s-t flow f that sends one unit of flow from s to t and satisfies all capacity constraints (i.e. $|f_e| \leq c_e$ for all $e \in E$). Write an LP that computes $d_{flow}(s, t)$.*

(c) *Prove that if $c_e = 1$ for all $e \in E$, then $d_{flow}(s, t)$ is the length of the shortest path in G from s to t.*
(Hint: let $d(s, t)$ be the length of the shortest path from s to t. You want to show $d_{flow}(s, t) = d(s, t)$ which is equivalent to showing that $d_{flow}(s, t) \leq d(s, t)$ and $d_{flow}(s, t) \geq d(s, t)$.)

Collaborators:

Solution: Your solution here!

Problem 3 (Road Trip 4: Road Trip Game) *Lorenzo is going on another trip, and he's taking Ruimin along with him. They've already figured out the shortest path to take, and the optimal gas stations to stop at along the way - all they need left is some entertainment. Thankfully, Lorenzo has thought of a great game to keep them entertained.*

The game is specified by an undirected, unweighted graph $G = (V, E)$ and two vertices $s, t \in V$. Lorenzo picks an edge $e \in E$, and Ruimin simultaneously picks a cut $S \subseteq V$ which must separate s and t (i.e. $s \in S$ but $t \notin S$). We define the edge boundary of a cut S to be the set of edges crossing the cut:

$$E(S, \overline{S}) := \{uv \in E \mid u \in S, v \in \overline{S}\}.$$

Lorenzo wins the game if the edge he picked is in Ruimin's cut's edge boundary, and Ruimin wins otherwise. Formally, for the selected e and S, Lorenzo's payoff $P(e, S)$ is given by

$$P(e, S) = \begin{cases} 1 & \text{if } e \in E(S, \overline{S}), \\ 0 & \text{otherwise.} \end{cases}$$

Lorenzo is trying to maximize $P(e, S)$, and Ruimin is trying to minimize it.
*We are interested in **mixed strategies** for this game: Lorenzo will assign each edge e some probability $p_e \geq 0$, and randomly select an edge accordingly. Similarly, Ruimin's strategy will involve randomly selecting a cut c, where each cut c is selected with probability p_c.*

(a) *Lorenzo knows that he can't outsmart Ruimin, so he knows that whatever strategy he picks, Ruimin is just going to read his mind and figure it out. To determine his best strategy, he is going to solve the following LP:*

$$\begin{aligned} \max_{\alpha, q} \quad & \alpha \\ & \sum_{e \in C} q_e \geq \alpha \quad \forall C \in \mathcal{C} \\ & \sum_{e} q_e = 1 \\ & 0 \leq q_e \quad \forall e \in E(G) \end{aligned}$$

where $\mathcal{C}$ is the set of s-t cuts in G. Explain, in words, how this LP represents the game.

(b) *Ruimin knows that Lorenzo is a cheater, so he knows that whatever strategy he picks, Lorenzo is going to sneakily take a glance at it before deciding on his own strategy. In that case, write the linear program that Ruimin must solve to determine his optimal strategy. (Assume that Lorenzo picks his strategy after Ruimin.)*

(c) *Demonstrate that these programs are each other's dual. (You may use the fact that the dual of a dual program is the primal.)*

(d) *Part (c) implies the existence of some probability p such that:*

- *If Lorenzo plays optimally, his probability of winning is at least p, even if Ruimin knows his strategy.*
- *If Ruimin plays optimally, Lorenzo's probability of winning is at most p, even if Lorenzo knows Ruimin's strategy.*

Provide a short argument (2 sentences) that part (c) implies this.

(e) *Sketch a proof that the p from part (d) is $p = \frac{1}{L}$ where L is the length of a shortest path from s to t. This is Lorenzo's expected payoff from the game if both players play optimally, and is known as the **value** of the game.*

Collaborators:

Solution: Your solution here!

You can choose to complete this theory problem **or** the programming assignment

PROBLEM 4 (SEQUENCE EDITING) *You are given a sequence of n integers, $a_1, \ldots, a_n$. At each time step, you may pick one of the n integers and either increase or decrease it by 1. You want to find the minimum amount of time to make the sequence non-decreasing.*

(a) Write a linear program in $2n$ variables that solves this problem.

Hint: *Notice that for each integer a_i, the optimal changes on a_i must be either increasing or decreasing it by some value. For each i, let p_i and m_i be variables that represent how much a_i increases and decreases respectively. What is the total change on a_i? For the sequence to be increasing after all changes, what constraints must be satisfied?*

(b) Find the dual of the above lienar program.

(c) Describe an efficient algorithm that solves the dual program. Sketch a proof of correctness.

Hint: *Try dynamic programming, and it should take time $O(n^2)$.*

Collaborators:

Solution: Your solution here!

You can choose to complete this programming problem **or** the previous theory assignment

PROBLEM 5 (MINESWEEPER) *Konstantinos* **really** *likes Minesweeper. The classic game was a great time sink back in Windows XP and very spotty internet. After reaching solving thousands of boards, it is time for the next step in the age of AI: Writing a program to solve Minesweeper for us.*

Minesweeper is played on an $R \times C$ board. Initially, information about all blocks is hidden. Clicking on a block either reveals how many mines are in the adjacent 8 blocks, or blows you up if you click on a mine. Your goal is to write an iterative solver which picks the safe blocks to click before receiving new information. You should use Linear Programming to achieve that by defining the following problem. For every block at coordinates (i, j), define a variable x_{ij} representing whether there is a mine in that block.

$$\begin{aligned} &\max 0 \\ &\text{s.t. } x_{ij} = 0 \ \forall \text{ clicked blocks} \\ &\sum_{xy \text{ neighbors of } ij} x_{xy} = value[i, j] \end{aligned}$$

You can find a working copy of the game to practice here. You will start work off this GitHub repository. You only need to fill in code in the `solver.py` *file, which is also the only file you need to submit.*

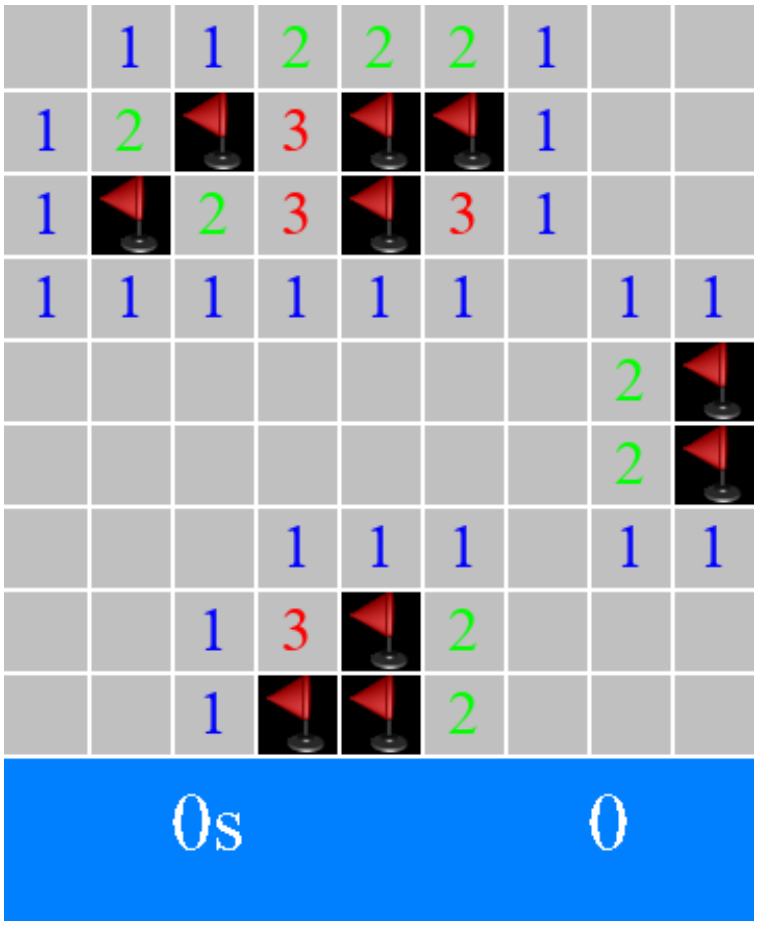

Figure 1: The small size is 9x9 with 10 mines in total.